# Jobsheet Praktikum: Pertemuan 7

## Overriding dan Overloading

| | |
|---|---|
| **Mata Kuliah** | Praktikum Pemrograman Berbasis Objek (RTI253008) |
| **Pertemuan** | 7 (Minggu 7) |
| **Durasi** | 1 × 4 × 50’ praktikum; 1 × 1 × 50’ tugas/laporan mandiri |

## A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Meng-override method warisan di dalam subclass, ditandai anotasi `@Override`, termasuk memanggil versi superclass lewat `super.method(...)`.
2. Membedakan overriding dari overloading dengan menulis method yang namanya sama tetapi daftar parameternya berbeda.

## B. Persiapan dan Prasyarat

- **Alat**: JDK 17 atau lebih baru, NetBeans (editor yang digunakan sepanjang praktikum ini).
- **Proyek**: jobsheet ini melanjutkan proyek `bank-mini` dari topik Inheritance. Proyek itu sudah berisi `Account`, `SavingsAccount`, `CheckingAccount`, `Customer`, `Bank`, dan `Main`.

> **Tanpa NetBeans?** Jobsheet ini tetap dapat diikuti menggunakan editor teks biasa:
>
> ```
> javac -d out src/id/ac/polinema/*.java
> java -cp out id.ac.polinema.Main
> ```
>
> Checkpoint dan output yang dihasilkan tetap sama persis, apa pun editor yang digunakan.

## C. Langkah Kerja

Diagram berikut adalah hasil akhir yang dituju jobsheet ini:

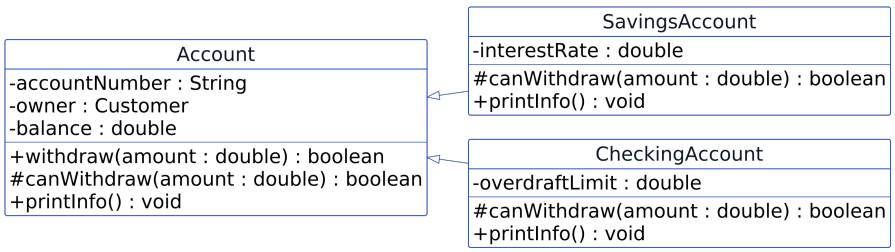



Figure 1: Diagram kelas Account, SavingsAccount, dan CheckingAccount setelah overriding

Cara membaca diagram:

- Tanda # berarti `protected`: boleh dipakai dan ditulis ulang oleh subclass.
- `canWithdraw()` dan `printInfo()` tertulis di `Account`, lalu tertulis lagi di kedua subclass. Method yang tertulis lagi di subclass berarti di-override.

> **Konsep Singkat: Overriding.** Bayangkan resep keluarga: seorang anak memasak hidangan dengan nama yang sama, tetapi memakai resepnya sendiri. Overriding bekerja seperti itu. Subclass menulis ulang method warisan dengan nama dan parameter yang sama, hanya isinya yang berbeda. Nama method ditambah daftar parameternya disebut **signature**.

### Langkah 1: Account Menyediakan Titik untuk Di-override

**Tujuan langkah ini:** memisahkan aturan “boleh menarik atau tidak” ke method tersendiri, supaya nanti tiap jenis rekening bisa punya aturannya sendiri.

Saat ini `withdraw()` di `Account` hanya tahu satu aturan: penarikan tidak boleh melebihi saldo. Atribut `interestRate` dan `overdraftLimit` di subclass belum berpengaruh apa pun.

1. Buka `Account.java`.
2. Tambahkan method baru `canWithdraw(double amount)` bertanda `protected`. Isinya aturan lama: jumlah harus positif dan tidak melebihi saldo.
3. Ubah `withdraw()` supaya memanggil `canWithdraw(amount)`.

```java
package id.ac.polinema;

public class Account {
    // ... (unchanged code omitted)
    public boolean withdraw(double amount) {
        if (!canWithdraw(amount)) {
            return false;
        }
        balance -= amount;
        return true;
    }

    protected boolean canWithdraw(double amount) {
        return amount > 0 && amount <= balance;
    }
    // ... (unchanged code omitted)
}
```

Figure 2: Account.java, withdraw() memanggil canWithdraw() yang baru

Jalankan program tanpa mengubah `Main.java`.

**Output yang diharapkan:**

```
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
```

**Mengapa demikian?** Output-nya sama seperti sebelum langkah ini. `canWithdraw()` masih berisi aturan lama, jadi perilaku program belum berubah. Langkah ini hanya menata ulang kode (refactoring) supaya langkah berikutnya bisa dilakukan.

**Checkpoint:** program berjalan dan output-nya sama dengan blok di atas.

⚠ **Jika gagal:** apabila muncul error `cannot find symbol: method canWithdraw`, periksa apakah nama method dan parameternya di dalam `withdraw()` sama persis dengan deklarasinya.

### Langkah 2: SavingsAccount Meng-override canWithdraw dan printInfo

**Tujuan langkah ini:** membuat rekening tabungan menjaga saldo minimum 50000, dan mencetak info yang lebih lengkap.

1. Buka `SavingsAccount.java`.
2. Tulis ulang `canWithdraw(double amount)` dengan anotasi `@Override`. Aturan barunya: saldo setelah penarikan tidak boleh kurang dari 50000.
3. Tulis ulang `printInfo()` dengan anotasi `@Override`. Panggil `super.printInfo()` lebih dulu, lalu cetak jenis rekening dan suku bunganya.

```java
package id.ac.polinema;

public class SavingsAccount extends Account {
    private static final double MINIMUM_BALANCE = 50000;
    private double interestRate;

    public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
        super(accountNumber, owner, balance);
        this.interestRate = interestRate;
    }

    public double getInterestRate() {
        return interestRate;
    }

    @Override
    protected boolean canWithdraw(double amount) {
        return amount > 0 && (getBalance() - amount) >= MINIMUM_BALANCE;
    }

    @Override
    public void printInfo() {
        super.printInfo();
        System.out.println("Account type: Savings, interest rate: " + interestRate);
    }
}
```

Figure 3: SavingsAccount.java meng-override canWithdraw dan printInfo

4. Perbarui `Main.java`:

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        boolean result = savings.withdraw(70000);
        System.out.println("Withdraw 70000 allowed? " + result);
        savings.printInfo();
    }
}
```

Figure 4: Main.java menguji penarikan yang melanggar saldo minimum

**Output yang diharapkan:**

```
Withdraw 70000 allowed? false
A003 - Rian - balance: 100000.0
Account type: Savings, interest rate: 0.02
```

**Mengapa demikian?**

- Penarikan 70000 dari saldo 100000 akan menyisakan 30000. Jumlah itu di bawah saldo minimum 50000, jadi `canWithdraw()` milik `SavingsAccount` mengembalikan `false`.
- Baris kedua dicetak oleh `super.printInfo()`, yaitu versi milik `Account`. Baris ketiga adalah tambahan dari `SavingsAccount`.
- `@Override` meminta compiler memeriksa bahwa method itu memang ada di superclass dengan signature yang sama. Salah ketik nama method langsung ketahuan.

**Checkpoint:** output program sama dengan blok di atas.

⚠ **Jika gagal:** apabila muncul error `method does not override a method from its superclass`, periksa apakah signature method di subclass sama persis dengan yang ada di `Account`.

### Langkah 3: CheckingAccount Meng-override canWithdraw dan printInfo

**Tujuan langkah ini:** membuat rekening giro boleh ditarik melebihi saldo, sampai batas `overdraftLimit`.

1. Buka `CheckingAccount.java`.
2. Tulis ulang `canWithdraw(double amount)` dan `printInfo()` dengan pola yang sama seperti Langkah 2. Aturan barunya: penarikan boleh sampai saldo ditambah `overdraftLimit`.

```java
package id.ac.polinema;

public class CheckingAccount extends Account {
    private double overdraftLimit;

    public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
        super(accountNumber, owner, balance);
        this.overdraftLimit = overdraftLimit;
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    @Override
    protected boolean canWithdraw(double amount) {
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }

    @Override
    public void printInfo() {
        super.printInfo();
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }
}
```

Figure 5: CheckingAccount.java meng-override canWithdraw dan printInfo

3. Perbarui `Main.java`:

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer1, 100000, 0.02);
        savings.withdraw(70000);

        Customer customer2 = new Customer("Dewi", "0812-0000-0004");
        CheckingAccount checking = new CheckingAccount("A004", customer2, 100000, 200000);
        boolean result = checking.withdraw(250000);
        System.out.println("Withdraw 250000 allowed? " + result);

        Bank bank = new Bank(10);
        bank.addAccount(savings);
        bank.addAccount(checking);
        bank.printAllAccounts();
    }
}
```

Figure 6: Main.java menguji penarikan melebihi saldo lewat overdraft

**Output yang diharapkan:**

```
Withdraw 250000 allowed? true
A003 - Rian - balance: 100000.0
Account type: Savings, interest rate: 0.02
A004 - Dewi - balance: -150000.0
Account type: Checking, overdraft limit: 200000.0
```

**Mengapa demikian?**

- Saldo A004 adalah 100000 dan batas overdraft-nya 200000. Penarikan 250000 masih dalam batas, jadi diizinkan dan saldonya menjadi -150000.
- `Bank.printAllAccounts()` tidak diubah sama sekali. Method itu tetap memanggil `printInfo()` untuk tiap rekening. Java menjalankan versi milik objeknya: versi `SavingsAccount` untuk A003, versi `CheckingAccount` untuk A004.

**Checkpoint:** output program sama dengan blok di atas, termasuk saldo negatif pada A004.

⚠ **Jika gagal:** apabila saldo A004 tidak pernah menjadi negatif, periksa apakah `canWithdraw()` di `CheckingAccount` membandingkan `amount` dengan `getBalance() + overdraftLimit`, bukan dengan `getBalance()` saja.

### Langkah 4: Overloading, deposit() dengan Catatan

**Konsep Singkat: Overloading.** Di kasir, satu kata “bayar” berlaku untuk tunai, kartu, dan QR. Kasir memilih caranya dari apa yang diserahkan. Overloading bekerja seperti itu: beberapa method memakai nama yang sama, tetapi daftar parameternya berbeda. Compiler memilih versinya dari argumen yang diberikan saat method dipanggil.

**Tujuan langkah ini:** menambah cara kedua untuk menyetor, yaitu setoran yang disertai catatan.

1. Buka `Account.java`.
2. Tambahkan method `deposit(double amount, String note)`. Method ini mencetak catatannya, lalu memanggil `deposit(amount)` yang sudah ada.

```java
package id.ac.polinema;

public class Account {
    // ... (unchanged code omitted)
    public boolean deposit(double amount, String note) {
        if (!deposit(amount)) {
            return false;
        }
        System.out.println(accountNumber + " deposit note: " + note);
        return true;
    }
    // ... (unchanged code omitted)
}
```

Figure 7: Account.java, deposit(double, String) sebagai overload dari deposit(double)

3. Perbarui `Main.java`:

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        savings.deposit(50000, "Initial top-up");
        savings.printInfo();
    }
}
```

Figure 8: Main.java memanggil deposit dengan catatan

**Output yang diharapkan:**

```
A003 deposit note: Initial top-up
A003 - Rian - balance: 150000.0
Account type: Savings, interest rate: 0.02
```

**Mengapa demikian?** Pemanggilan `deposit(50000, "Initial top-up")` membawa dua argumen, jadi compiler memilih versi dua parameter. Versi itu mencetak catatan, lalu menyerahkan penambahan saldo ke `deposit(double)`. Kedua versi tetap bisa dipakai.

**Checkpoint:** output program sama dengan blok di atas.

⚠ **Jika gagal:** apabila muncul error `reference to deposit is ambiguous`, periksa apakah kedua method `deposit` benar-benar berbeda daftar parameternya (jumlah atau tipe), bukan hanya berbeda nama parameter.

## D. Tugas dan Hasil Kerja

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program setelah Langkah 4.
- **Tugas mandiri:**
  1. Terapkan pola overriding yang sama pada `BusinessAccount` yang kamu buat pada topik Inheritance. Override `canWithdraw(double)` agar penarikan wajib menyisakan saldo minimum 1000000, dan override `printInfo()` agar ikut mencetak jenis rekening. Diagram berikut hanya sketsa method yang perlu ditulis ulang, BUKAN kode jadi, isinya diserahkan sepenuhnya padamu:

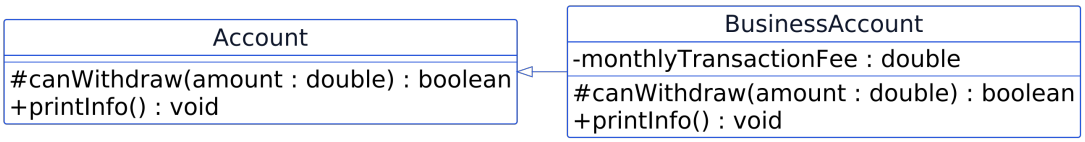



Figure 9: Sketsa BusinessAccount yang meng-override canWithdraw dan printInfo

Buktikan lewat `Main.java`: buat satu `BusinessAccount` bersaldo 2000000, coba tarik 1500000, lalu cetak infonya. Hasilmu benar apabila:

- penarikan 1500000 ditolak;

- `printInfo()` mencetak baris milik `Account`, diikuti baris jenis rekening;
- kedua method memakai `@Override`.

2. Jawab secara singkat (1-2 kalimat untuk masing-masing pertanyaan):
   - (a) Apa perbedaan signature antara method yang di-override (`canWithdraw()`, `printInfo()`) dan method yang di-overload (`deposit()`)?
   - (b) Siapa yang menentukan versi method yang dijalankan pada overriding, dan siapa pada overloading?

## E. Kriteria Penilaian

| Komponen | Bobot | Kriteria Lengkap (100%) | Kriteria Minimum |
|---|---|---|---|
| Langkah kerja tuntas | 40% | Seluruh langkah dijalankan dan berfungsi | Sebagian besar langkah selesai, hasil akhir berjalan |
| Checkpoint terverifikasi | 35% | Semua checkpoint tercapai dan dibuktikan (screenshot/output) | Sebagian checkpoint terbukti |
| Tugas mandiri | 25% | Override `BusinessAccount` benar dan jawaban konsep tepat | Override ada meski jawaban belum lengkap |